Starfront distributed live stacking

Rapid Stacker Collab's AWS backend ported into a LAN-hosted Rust hub, volunteer worker machines on the local network, one shared plate-solving catalogue on a network folder, and live-stacking progress pages on the Starfront website. Written 2026-10-22 for the author's review; the decisions table records what was settled that day, the open questions what was not.

Context

Rapid Stacker Collab (RSC) is live at `collab.astro.ekeservices.com` on its own AWS stack: 65 route Lambdas and 6 internal ones over DynamoDB, S3 and SQS, with pull-based Rust workers (`collab-worker`) on the author's Mac and in an EC2 pool. It costs about $155 to $230 a month, every artefact crosses the internet twice, and the people who would volunteer compute are the imagers themselves, on their own LANs. Starfront already carries the other half of the story: its N.I.N.A. plugin (1.0.0.33) embeds RSC's upload engine and uploads a collaboration's lights to the cloud stack when the coordinator puts a join code on the Starfront project.

The author wants the stacking backend brought home: a hub that runs on one machine of a local network, workers that volunteers add to that network with one click, a single copy of the Gaia plate-solving catalogue on a shared folder, and the progress pages that today live only on the RSC website moved to `collab.starfront.space`, so people enrolled in a collaboration can watch their stacks grow where they already sign in. The cloud stack stays as a second backend behind the same contract.

The outcome is: a coordinator binds a Starfront collaboration to a hub instead of a cloud join code; every telescope on it uploads as it does today; the hub pulls the frames down, deals them to LAN workers, and pushes previews, status and masters back; the website shows the project; and a Starfront user turns a spare PC into a worker from the desktop program without touching a terminal. All of it must be testable on the author's home network with no cloud account.

Decisions taken with the author (2026-10-22)

Question	Decision
Where raw frames land	**`collab.starfront.space` by default.** The server is a durable mailbox; the LAN hub pulls frames down and pushes artefacts up, so no hub owner exposes a port. A telescope on the hub's own LAN may upload straight to the hub. Everything must also run entirely on the author's home network (a "local mode").
What the hub is	**A Rust Axum service**, crate `worker/crates/hub` (binary `collab-hub`) in the RSC repo, beside `collab-engine` and `collab-worker`, sharing their record types, `Store` / `ObjectStore` traits, `DirStore` semantics, `collab-filters`, the status schema and the fake ports. Starfront (public, MPL-2.0) carries only the integration: the contract, launching, configuration, the volunteer UI, and the web pages. Hub and worker binaries are downloaded from EKE, as the worker is today.
Web pages	**Progress pages plus "My projects"**: the public project page, per-filter and combined previews, deep zoom, contributors and rigs, timeline, and a Discord-cookie dashboard of the projects a user is enrolled in with their own frame verdicts. Owner tools (frame editor, members, policies, gold, delete) stay in the desktop Collab tab, so the server's rule that admin tokens never reach a browser holds.
Engine	**The Collab Rust engine** (`collab-engine` over rapid-stacker `collab-api-3`). Starfront's Python tile stacker (`astrocontrol/livestack.py`) stays as the single-rig local preview; the orphaned, never-mounted `server/stacks.py` is removed.
Volunteer agent	**The Starfront desktop program plus a CLI.** A "Volunteer this machine for live stacking" section fetches the signed worker build, installs and supervises it; `collab-worker enrol <hub> --code …` does the same on headless Mac and Linux boxes. One enrolment protocol.
The AWS stack	**Keeps running** as a second backend behind the same contract. A Starfront project chooses its backend; the worker keeps its `aws` feature; nothing in the Terraform stack changes except a config name list.

Findings that shape the design

- **The worker already has the seam.** `collab-worker`'s `DdbStore` (`worker/crates/worker/src/daemon/store.rs`) implements the engine's `Store` and `StreamLeases` traits over its DynamoDB port, and `daemon/local.rs` already implements the blob and queue ports as JSON over HTTP (loopback only). A `HubStore` over HTTP replaces `DdbStore` one for one; the daemon's handlers (`prepare.rs`, `accumulate.rs`, `gold.rs`, `jobs.rs`) do not change.
- **The engine already has a reference store.** `collab_engine::store::local::DirStore` (directory plus one lock, 2 591 lines) implements every transition `simulate` needs and is exercised by the engine's kill-at-every-boundary tests. The hub is that store with a database and a network, not a re-port of the Lambdas from scratch.
- **The Lambdas' worker-route rules already exist in Rust**: `daemon/fake/api.rs` holds the five worker routes over `DirStore` for the daemon tests. Made generic over `&dyn Store`, they are the hub's worker routes.
- **Plate solving needs no ASTAP and no index files.** The engine solves with header hints against a per-project cone search of the PixInsight Gaia DR3 XPSD files (the 16 `gdr3-*.xpsd` band files, about 45 GB of the author's 108 GB `~/Astronomy/XPSD`), cached as `catalog-{project}.json`. rapid-stacker's XPSD reader uses plain `File::open` with seeks, no memory mapping, no locks and no writes, so one read-only copy on an SMB share is safe for any number of readers. The cost is that each cone search re-walks every band file, which is why the hub, not every worker, should run it. `COLLAB_ASTAP_PATH` is parsed and unused; Starfront's own `astrocontrol/solving.py` is the only ASTAP caller and stays that way.

- **Throughput is not the reason for a fleet.** Prepare plus accumulate plus gold takes about 2.2 s per 26 MP frame on an M3 Max, CPU only; one such machine clears a 5 000-sub night in about three hours. Volunteers buy resilience (a hub whose owner's machine is imaging) and short gold runs, not raw capacity. The plan sizes defaults accordingly (small scratch, pause while imaging).
- **Starfront's server is the right mailbox.** It already has agent and user tokens, Discord device-code sign-in, a web cookie for `/files`, a streaming `PUT` upload with magic-byte checks and a free-space floor, a plugin manifest feed and an installer download route, SQLite with payload-JSON rows and in-code migrations, and a systemd plus Caddy deploy. The plan extends those patterns rather than adding new ones.
- **The Starfront plugin needs no Core change.** `RapidStackerCollab.Core` is a verbatim copy (`SOURCE.md`); its `ApiClient` takes a base URL and a token string. Per-project routing, a grant token store and the LAN short-cut all live in Starfront's `Services/Stacking/`.
- **Nothing in Starfront runs CI** (its workflows sit under `nina/.github/`, which GitHub does not read). The plan moves them.
- **Two licence lines meet.** `worker/` in RSC is proprietary; the Starfront repo is MPL-2.0. The hub stays in RSC; what crosses into Starfront is the contract (documentation), the static pages (the author owns them and can dual-license; flagged), the alias table (already in Starfront's Core copy under MPL) and signed binaries as downloads. The XPSD files are the user's own PixInsight download and are never redistributed.

---

0. Design decisions this plan takes

0.1 Worker-to-hub adapter: the engine's `Store` / `ObjectStore` over HTTP, not the DynamoDB ports

	A. HTTP versions of `TablePort` / `BlobPort` / `QueuePort` (the `local.rs` shape)	**B. Engine `Store` / `StreamLeases` / `ObjectStore` over HTTP (the `DirStore` shape)**
Worker code	`local.rs` reused almost as-is; `DdbStore` unchanged	new `HubStore` (typed calls, about 500 lines) and `HubObjects`; `DdbStore` stays for AWS
Hub must implement	a DynamoDB emulator over SQLite: the strict expression engine (`fake/expr.rs`, 1 174 lines), four GSIs, document items	the engine's `Store` trait over SQLite, a port of `DirStore` whose semantics the engine's suites already test
Hub's own routes (frames init, result, checkpoint, sweep…)	a second port of every Lambda against a document store	written against the same `SqliteStore` the RPC exposes: one implementation of each transition
Wire protocol	DynamoDB expression strings on the wire	typed JSON ops; `child.rs` already defines this protocol (`{"op": "get_frame", …}` over the prepare child's pipe) and it covers every call the engine makes
Fencing and conditions	evaluated by the expression engine from strings the worker composed	evaluated in Rust on the record, as `DirStore` does

B. The hub never has to be a DynamoDB. `ApiPort` stays the production `HttpApi` (`daemon/aws/api.rs`) pointed at the hub's contract-conformant `/api/worker/*`, so the hub is held to `docs/api/openapi.yaml` by the same check as the Lambdas. The closed list of direct writes (open point 41: T4, T4b, T4c, T4d, T10, T13, T16, the photometric anchor, `tier1_*`) and every read go over the RPC; result, checkpoint, lease and progress keep going through `ApiPort`; queues get a small HTTP adapter (receive, delete, visibility, send) that `local.rs`'s `LocalQueues` already is, with the loopback rule relaxed to the configured hub origin. Phase 1 makes the engine's store tests generic so `DirStore` and `SqliteStore` run one suite.

If the SQLite port stalls, the fallback is to embed `DirStore` itself in the hub behind the same RPC (its in-process lock is valid in one server process) and move records into SQLite later behind the same trait. The route-side transitions (T1, T2, T3, T11, T12, jobs) are needed either way.

0.2 SQLite crate: `rusqlite` (bundled), a recorded deviation from the Apps/Rust convention

The author's Axum services use runtime `sqlx`, repositories returning `anyhow::Result`, inline `CREATE TABLE IF NOT EXISTS` migrations, an `AppState` of `Arc`s, a custom `AppError: IntoResponse`, `tracing_subscriber` with `EnvFilter`, security headers as the last layer, and no inline scripts. The hub keeps all of that except the driver:

1. The `Store` trait is synchronous and is called from engine threads and from `spawn_blocking`; every transition is read-then-conditional-write inside one `BEGIN IMMEDIATE` transaction. `rusqlite` expresses that directly; `sqlx`'s async SQLite driver makes a sync `Store` awkward.
2. Rows are documents, as in `DirStore` and in Starfront's `store.py`: the engine's serde struct in a `doc` column plus the indexed columns. Compile-time query checking buys little.
3. `bundled` compiles SQLite in, so a volunteer's Windows or macOS hub needs no system library.

Shape: one writer connection behind a mutex, a small pool of readers, WAL, `synchronous=NORMAL`, `busy_timeout` 5 s; migrations as `include_str!` SQL files recorded in `_migrations`. The hub's README records the deviation so nobody "fixes" it.

0.3 Auth: the Starfront server mints hub-scoped grants; the hub verifies them offline

Chosen over "the hub asks the server to verify each agent token" because a hub behind NAT must keep serving its LAN when the VPS is unreachable.

- **Registration.** From the desktop Collab tab (which holds the owner's Discord `userToken`), `POST /api/v1/hubs {name, lanUrl}` answers `{hubId, hubToken (sfh_ + 64 hex, shown once), grantSecret (32 bytes, shown once)}`. The hub keeps both in a 0600 credentials file. The secret is per hub: a compromised hub can forge grants only for itself.
- **Grants.** The server's `GET /api/v1/agent/projects` listing gains a `stacking` object per project (4.3). For a hub-backed project it carries `grant`: `sfg_` + base64url(`{"v":1,"hub","sub","name","agent","project","iat","exp": iat+86400}`) + `.` + base64url(HMAC-SHA256 under `grantSecret`). Python's stdlib `hmac` signs; the hub verifies with the `hmac` crate. The plugin re-reads the listing at every check-in, so a removed member stops within one poll; the hub also applies membership revocations it pulls from the server (4.2), which is the real cut-off.
- **Use.** The plugin presents `Authorization: Bearer sfg_…` to the server's `/stack/api/*` and, on the LAN short-cut, to the hub's `/api/*`. The hub's auth layer accepts three bearer kinds: `rscw_` (workers), `rscu_` (hub-minted tokens for `collab-uploader` and scripts, minted on the admin

surface; no device flow in v1), `sfg_` (grants). Core's `ApiClient` only needs a string from `ITokenStore`, so no Core change.

- **Cloud projects** keep the `rscu_` device flow unchanged.
- **Identities.** A hub `user_id` is a UUID v4 (the contract's `UserId`) minted once per Starfront user and kept in `identities`, so no Discord id appears in a Collab record, key or `status.json`.

HMAC rather than Ed25519 for grants because the Starfront server has no `cryptography` dependency (`install.sh` installs `fastapi`, `uvicorn`, `numpy`) and a per-hub secret gives the same containment. Ed25519 is used where parties share no secret: the worker-binary manifest (4.6).

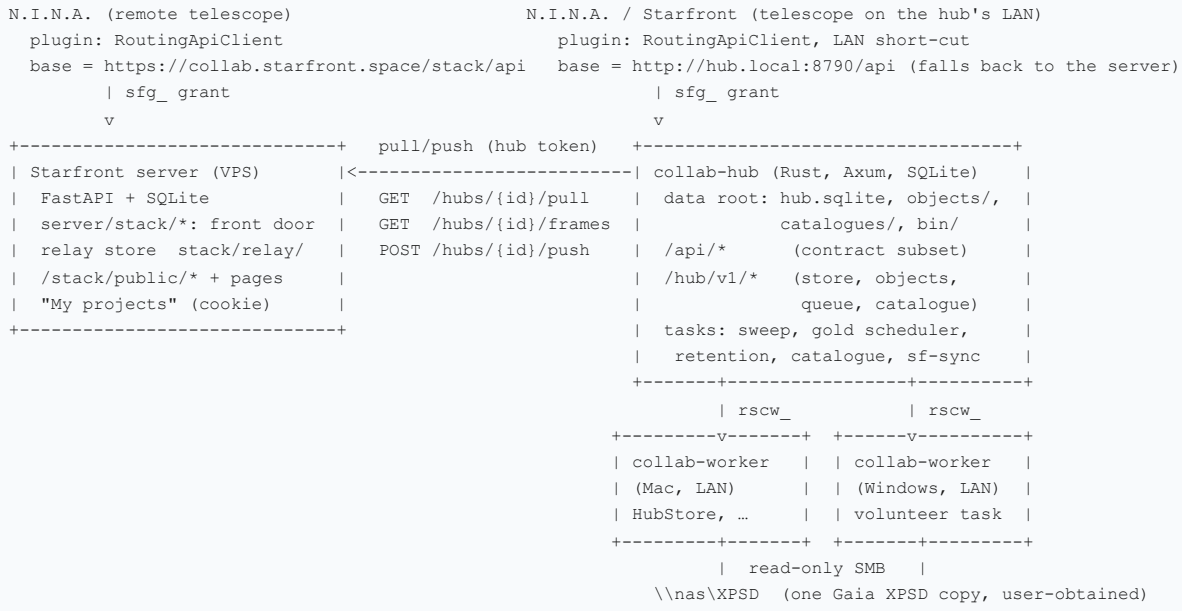
0.4 Ownership of state between the Starfront server and the hub

State	Owner	Direction
Starfront project, its `stacking` binding, Discord identities, telescope agents	Starfront server	hub pulls
Collab project row (id, slug, grid, policy, filters, status, join code), members, rigs, filter mappings, terms acceptance, library frame rows	Starfront server for hub-backed projects (it is the identity authority and the plugin's front door; hub projects are `approval: open` because the collaboration already gates membership)	hub pulls a cursor-based change feed
Grid changes after the first light (`grid_rev`), streams, sessions, jobs, queues, leases, frame rows from `uploaded` onward (status, verdicts, metrics, transforms)	hub	hub pushes verdict summaries and the grid back
Raw frames	uploaded to the server (default) or the hub (LAN); the server keeps bytes until the hub has acknowledged them, plus a retention window	hub pulls bytes by `Range`
`status.json`, previews, deep zoom, history, masters	hub (workers write them into the hub's object root)	hub pushes files; the server serves `/stack/public/{project_id}/…`

The server is a durable mailbox in both directions; the hub never needs an inbound connection; the LAN keeps working when the VPS is unreachable (only the public pages lag). The price is that the contributor subset of the contract exists twice, in Rust on the hub and in Python on the server; the contract check holds both.

---

1. Architecture



Local mode is the same picture with the VPS replaced by `python server/run_local.py --lan` on the Mac (7.1).

2. Phase map

Phase	Area	Deliverable	Proves
0	contract	hub profile, open points, crate scaffold, CI	the shape is agreed before code
1	hub core	`SqliteStore`, objects, queues, worker routes, RPC	`daemon_e2e` reproduces `simulate` through the hub
2	worker	`hub` adapters, `enrol`, capabilities, version handshake, service install	two real workers on the LAN drain a hub
3	hub complete	frame ingress, contributor routes, status publishing, admin and enrol surface, background tasks, catalogue service	`collab-uploader` → hub → `status.json`, no Starfront involved
4	Starfront server	hubs, binding, grants, front door, relay, push ingestion, pages, dashboard, manifest hosting	remote plugin → server → hub → pages
5	Starfront desktop	hub wizard, volunteer feature, supervisors, checks	a non-technical volunteer installs a worker from the desktop
6	N.I.N.A. plugin	backend per project, routing client, LAN short-cut, checklist	a real night on the author's rigs
7	local mode, coexistence, CI	the home-network runbook and scripts, migration notes, docs	the whole thing, end to end, repeatably

Each phase ends with a verification list that needs nothing from a later phase.

Phase 0: contract and scaffolding (RSC)

0.A Contract changes (`docs/api/`, landing with the code of each later phase; `info.version` 1.27.0, additive)

New open points in `docs/api/README.md`, numbered from 109:

- **109. A second backend behind the same contract: the LAN hub.** The "hub profile" (`docs/api/hub-profile.json`): the worker routes (5); the contributor routes the plugins use (`projects/resolve-join-code`, `projects/{p}`, `join`, `membership` GET/PUT, `rigs` GET/POST/PATCH, `filters/normalize`, `frames/init|complete|list|get|withdraw`, `rigs/{rig}/library*`, `projects/{p}/status`, `projects/{p}/masters`, `projects/{p}/events`); the two static files; the admin routes it keeps (`admin/fleet`, `admin/pause`). Not implemented by a hub: `auth/*`, `tokens`, `device/*`, `projects` POST/PUT (projects come from Starfront), `members` PATCH/DELETE, `mosaic`, `projects/{p}/delete|restore`.
- **110. Bearer kinds.** `sfg_` grants, accepted by a hub and by the Starfront front door; `rscu_` / `rscw_` unchanged. Never 403 or 404 under `/api/*` on the hub either.
- **111. Heartbeat carries `contract` and the catalogue state; the response carries `min_contract`.** `Capabilities` gains optional `contract`, `catalogue` (`ready|missing|unreadable|stale`), `catalogue_path_ok`, `idle_policy` (`always|when_idle`), `busy`. A hub answers 409 `contract_mismatch` to a worker below `min_contract`; the worker drains and exits 3 so a supervisor does not flap.
- **112. Part URLs may be relative or same-origin.** Clients already treat them as opaque; the hub and the server sign nothing beyond the 32-byte `upload_id` in the path.
- **113. The catalogue endpoint** `GET /hub/v1/projects/{p}/catalogue?grid_rev=N` (hub-private) and the rule: a worker uses the hub's `catalog-{project}.json` for the current `grid.spec()`, else builds it locally and `PUT`s it back.
- **114. Hub to Starfront sync** is Starfront's `/api/v1/hubs/*` API, documented in Starfront's README; the records that cross it are `records.md` shapes.

`queues.md` gains "On the hub" (same bodies, attributes and semantics over SQLite). `s3-layout.md` gains "On the hub: the same keys under `<data root>/objects/`" and the push layout on the server.

0.B Crate scaffold

- `worker/Cargo.toml`: `members = ["crates/*"]` picks the crate up. Workspace deps to add: `axum 0.8`, `tokio` (full), `tower-http` (`fs`, `trace`, `set-header`, `limit`), `rusqlite 0.37` (`bundled`, `serde_json`), `hmac`, `sha2` (present), `ulid`, `rand`, `hex`, `mime_guess`, `sysinfo`. `[lints] workspace = true` in the new crate (omitting it silently drops `unsafe_code = "deny"`).
- `worker/crates/hub/Cargo.toml`: `collab-hub`, lib `collab_hub` + binary; depends on `collab-engine`, `collab-filters`, `collab-worker` (lib: the ports' request types, `fake::api` after its refactor, `packed.rs`, `capabilities`).
- A shared `wire` module rather than a fourth crate: move the typed request and response structs of `daemon/ports.rs` and the op enums of `daemon/child.rs` into `collab-worker/src/wire.rs`, re-exported from both, so the hub and the worker share one definition. A `collab-wire` crate is the cleaner cut once a second consumer exists.

0.C `scripts/check-contract.mjs`: the hub component

`scripts/lib/hub.mjs` reads `worker/crates/hub/src/routes/*.rs` for `.route("…")` strings and error codes (as `lib/website.mjs` reads `api.js`), compares them with `hub-profile.json` and the contract, and fails on a route the profile names that the hub lacks, a route the hub has that the profile does not name, or a code `ErrorCode` lacks. A drive-for-real step spawns `cargo run -p collab-hub -- simulate --port 0` (3.7), calls every profile operation with the website mock's bodies and validates answers with the Lambdas' validator from `backend/lambdas/shared/contract.json`; skipped with a note when `cargo` is absent.

0.D CI

`worker-ci.yml` already builds every workspace crate on three OSes; add a `hub-e2e` job (ubuntu, macos) running `scripts/hub-e2e.sh --only one` (Phase 3). `contract-ci.yml` runs the hub component. Nothing needs AWS.

Verify: `npx @redocly/cli lint docs/api/openapi.yaml`; `node scripts/check-contract.mjs` (the hub component warns "0 of N profile routes" until Phase 3, not fails); `cargo build --workspace` on all three OSes.

Phase 1: the hub core (RSC, `worker/crates/hub/`)

1.1 Module layout

```
worker/crates/hub/src/
  main.rs            clap: run | check | simulate | enrol-code | token | migrate | import | backup | status
  config.rs          HubConfig: collab-hub.toml + COLLAB_HUB_* (env wins), validation, the `check` report
  app.rs             AppState { store, objects, queues, auth, catalogue, clock, cfg }, Router, layers
  error.rs           HubError -> {ok:false, code, message}; status mapping from the contract's error table
  auth.rs            bearer kinds (rscw_/rscu_/sfg_), enrol codes, token table, HMAC grant check
  db/mod.rs          Db { writer: Mutex<Connection>, readers }, with_tx(BEGIN IMMEDIATE)
  db/migrations/0001_init.sql …
  store/{mod,frames,streams,jobs,queries}.rs   SqliteStore: Store + StreamLeases (the DirStore port)
  objects.rs         FsObjects = collab_engine::store::local::DirObjects over <root>/objects (reused as-is)
  queue.rs           SqliteQueues: QueuePort semantics (visibility, receipts, delay, DLQ, attributes)
  routes/worker.rs   heartbeat, lease, result, checkpoint, progress (fake/api.rs made generic)
  routes/rpc.rs      /hub/v1/store/{op}, /hub/v1/objects/{key}, /hub/v1/queue/{op}, /hub/v1/.../catalogue
  routes/frames.rs   frames init|complete|list|get|withdraw, parts PUT, library/*       (Phase 3)
  routes/projects.rs resolve-join-code, project, join, membership, rigs, status, masters, events (Phase 3)
  routes/admin.rs    enrol codes, fleet, pause, projects, worker-manifest mirror          (Phase 3)
  routes/public.rs   /public/{project_id}/... with the s3-layout.md cache headers         (Phase 3)
  tasks/mod.rs       sweep, gold_scheduler, retention, catalogue, starfront_sync         (Phase 3/4)
  catalogue.rs       cone-search service and cache                                       (Phase 3)
  starfront.rs       client of the Starfront server; publish.rs: what to push            (Phase 4)
```

Conventions: handlers → services (`store`, `queue`, `catalogue`) → `db`; `AppState: Clone` of `Arc`s; `HubError: IntoResponse` with the contract envelope; `tracing_subscriber` with `EnvFilter` (`collab_hub=info,tower_http=warn`), `--log-format json` for the desktop's log pane; the last layer sets `X-Content-Type-Options`, `X-Frame-Options`, `Cache-Control: no-store` on `/api`. The hub serves no HTML beyond a one-line health page.

1.2 `config.rs`

`collab-hub.toml` in the config dir (`~/.config/collab-hub/`, `%APPDATA%\collab-hub\`, or `--config`):

```
bind = "0.0.0.0:8790"                    # COLLAB_HUB_BIND; loopback by default under `simulate`
public_url = "http://hub.local:8790"     # what enrol hands to workers and the LAN short-cut uses
data_root = "/Volumes/Stack/collab-hub"  # COLLAB_HUB_DATA_ROOT: hub.sqlite, objects/, catalogues/, bin/, logs/
xpsd_dir = "/Volumes/XPSD"               # COLLAB_HUB_XPSD_DIR, read-only
min_contract = "1.27.0"

[xpsd_paths]                             # what enrol hands a worker of each OS
macos = "/Volumes/XPSD"
windows = '\\nas\XPSD'
linux = "/mnt/xpsd"

[starfront]                              # Phase 4; absent = standalone hub
url = "https://collab.starfront.space"
hub_id = "hub_…"                         # token and grant secret live in hub-credentials.toml (0600)

[tls]                                    # optional: self-signed cert from `collab-hub check --make-cert`
cert = ""
key = ""

[limits]
free_gb_floor = 50
```

`collab-hub check` prints every setting found, missing or invalid, probes `data_root` for writability and free space, lists `xpsd_dir` (`gdr3-*.xpsd` count and size; opens one band file as `cone_search_local` would) and exits non-zero on a problem. It is the service's `ExecStartPre` and the desktop wizard's "Check" button.

TLS on a LAN is **off by default**: the bearer token is the secret, and raw subs on a home LAN are not confidential (the plugin strips observer location before upload). `--tls` with a self-signed certificate whose SHA-256 fingerprint the enrol answer carries, pinned by the worker. A hub exposed beyond the LAN sits behind Caddy.

1.3 `db/`: schema mirroring `records.md`

One table per logical table, `doc` JSON holding the engine record, indexed columns extracted on write:

```
projects (project_id PK, slug UNIQUE, join_code UNIQUE, owner_id, visibility, status, grid_rev, updated_at, doc)
members  (project_id, user_id, role, status, display_name_lower, doc, PK(project_id,user_id),
          UNIQUE(project_id, display_name_lower))
rigs     (user_id, rig_id, fingerprint, doc, PK(user_id,rig_id))
frames   (pk, frame_id, project_id, status, project_status, user_id, uploaded_at, project_sha256,
          updated_at, ttl_epoch, doc, PK(pk,frame_id))
  INDEX frames_status (project_status, updated_at); INDEX frames_hash (project_sha256);
  INDEX frames_user (user_id, uploaded_at)
init_sentinels (project_id, user_id, sha256, ref_pk, ref_frame_id, created_at, ttl_epoch, PK(project_id,user_id,sha256))
sessions (project_id, session_id, doc, PK(...));   streams (project_id, stream_key, doc, PK(...))
jobs (job_id PK, project_id, state, created_at, doc); INDEX jobs_project (project_id, created_at)
events (project_id, ts_id, doc, ttl_epoch, PK(...))
tokens (token_hash PK, token_id, kind, user_id, scopes, hint, created_at, expires_at, revoked_at, doc)
workers (worker_id PK, doc, last_heartbeat_at, ttl_epoch, drain)
library (owner, sk, doc, raw_delete_after, PK(owner, sk))
identities (starfront_user_id PK, user_id UNIQUE, display_name, updated_at)
enrol_codes (code_hash PK, kind, created_at, expires_at, used_at, note)
uploads (upload_id PK, raw_key, frame_id, created_at); upload_parts (upload_id, part_number, size, etag, PK(...))
queue_messages (id PK, queue, msg_id, body, attrs, visible_at, receive_count, receipt, enqueued_at, expires_at)
  INDEX queue_visible (queue, visible_at)
settings (key PK, value)     -- pause flag, starfront cursor, hub identity
_migrations (version PK, applied_at)
```

`ttl_epoch` rows are reaped by the retention task. Every status write rewrites `project_status` (`{project_id}#{status}`) so the `StatusIndex` queries (`frames_with_status`, the sweep's "uploaded older than 10 min", the checkpoint route's `stillPending`) are index scans.

1.4 `store/`: `SqliteStore` as a port of `DirStore`

Port method by method from `worker/crates/engine/src/store/local.rs`, keeping its names and comments, replacing "one lock plus files" with "one `BEGIN IMMEDIATE` transaction plus rows":

`Store` method	Reference
reads (`get_project`, `get_member`, `get_rig`, `get_frame`, `frames_by_hash`, `user_frames`, `library_frames`, `frames_with_status`, `get_session`, `get_stream`, `project_frames`, `list_*`)	`DirStore`; reader connection; `frames_with_status` is `WHERE project_status = ?`
`begin_prepare` (T4), `begin_refresh` (T4c), `release_prepare` (T4b/T4d), `resume_frame` (T13), `refresh_frame` (T16)	`DirStore`; conditions evaluated on the deserialised `FrameRecord`
`complete_prepare` (T5–T8, T8r, T14, T15, T17)	`DirStore::complete_prepare` plus the Lambda-only parts in `backend/lambdas/shared/prepare-results.js` (T17 requeue on `grid_rev`, rotation-from-first-light, `retireIfEmptied`, the gold threshold trigger, the debounced calibration `reprepare` job of open point 48, the `accumulate` send); `fake/api.rs` already adds most of these over `DirStore`
`set_photometric_anchor_if_absent`	`DirStore`
`commit_checkpoint`, `record_tier1`, `mark_in_gold` (T10), `publish_gold`, `set_stream_preview`, `reserve_gold_seq`	`DirStore`, `worker-stream-checkpoint/index.js` (mark-only and preview-only forms); the pointer lands first, as in DynamoDB
`StreamLeases`	`collab_engine::lease` + `DirStore`; `lease_epoch` never removed

Add the route-side transitions `DirStore` lacks because `simulate` seeds records directly: T1 (init with sentinel and quota reservation), T2/T2b (complete), T3 (abort uploading), T11 (withdraw), T12 (remap), job creation (`createGoldJob` with the `active_gold_job` guard), member and rig upserts, events. Each is a port of the named Lambda (`frames-init`, `frames-complete`, `shared/frames.js abortUploading`, `shared/withdrawal.js`, `filters-remap`, `shared/gold-jobs.js`, `shared/rigs.js`, `shared/event-log.js`).

Agreement with `DirStore`: `worker/crates/engine/src/test_support.rs` gains `store_conformance<S: Store>(make: impl Fn() -> S)` holding the assertions of `engine/tests/{lease,accumulate,pipeline_gold}.rs` that only need a `Store` (fault plans through a `FaultStore` wrapper rather than `DirStore::fail_next_commit_after_pointer`). `collab-hub/tests/store_conformance.rs` runs it over `SqliteStore`; `tests/record_fixtures.rs` loads `docs/api/examples/records/engine/*.json` and reads them back byte-identical.

1.5 `objects.rs`

Reuse `collab_engine::store::local::DirObjects` over `<data_root>/objects/` unchanged. Routing by prefix as `daemon/objects.rs` does (`public/` served by `routes/public.rs`, `private/` only through signed URLs). Workers pack `reg/` FITS (`daemon/packed.rs`) before `PUT`; the hub stores bytes as given. `refresh_read` is a no-op on the hub (no lifecycle expiry); the retention task prunes `reg/` by the 45-day rule only below the free-space floor (3.5).

1.6 `queue.rs`

Port the semantics of `daemon/fake/mod.rs` (`Msg`, visibility, receipts, redrive, delays, attributes) and its tests onto SQLite. `receive(queue, max, wait_s)`: under `BEGIN IMMEDIATE`, select visible messages oldest first, set `visible_at = now + visibility`, bump `receive_count`, issue a new receipt; at 5 receives move to `<queue>-dlq`. A `tokio::sync::Notify` per queue wakes long-pollers on `send`; the handler holds the request up to 20 s. `delete` needs the current receipt (`ReceiptHandleIsInvalid` → 400 with the AWS error name so `aws/ddb.rs::classify` keeps working); `change_visibility(0)` makes it visible now; `send` honours `delay_s` (≤ 900) and attributes. Retention 4 d main, 14 d DLQ. Attributes `type`, `project_id`, and `min_ram_gb`, `min_scratch_gb`, `prefer_gpu: "false"`, `skips` on `accumulate` and `gold`, computed where the Lambdas compute them (`shared/messages.js`, `shared/gold-jobs.js`).

1.7 `routes/worker.rs` and `routes/rpc.rs`

- Refactor `daemon/fake/api.rs` so its five route functions take `&dyn Store` plus a `RouteCtx` (clock, queue sender, gold trigger) instead of `&FakeAws`; the fake keeps calling them over `DirStore`, the hub over `SqliteStore`. One source of the Lambdas' rules in Rust. Heartbeat upserts `workers` and answers `ingest_paused`, `drain`, `next_heartbeat_s`, `min_contract`, `server_time`; 409 `contract_mismatch` when below.
- RPC `/hub/v1/store/{op}` (worker token): one POST per op, bodies and answers in the `child.rs` pipe shapes (`Request{id, call}` → `{"ok": …}|{"err": …}`). Ops: every `Store` read, T4/T4b/T4c/T4d/T10/T13/T16, `set_photometric_anchor_if_absent`, `record_tier1`. Anything else is 400 `invalid_request` (the closed list of open point 41).
- Objects `/hub/v1/objects/{key}`: `GET` (with `Range`), `HEAD`, `PUT` (streamed to a temp file then `rename`), `DELETE`, `GET /hub/v1/objects?prefix=` listing: the `s3-http.js` surface that `LocalBlobs` already speaks. Keys checked with `is_safe_key`; `public/` writes only by workers; `raw/` writes refused (contributors use multipart; the relay task writes it).
- Queue `/hub/v1/queue/{op}`: `ReceiveMessage`, `DeleteMessage`, `ChangeMessageVisibility`, `SendMessage` with the inputs `LocalQueues` sends.

1.8 Tests

`collab-hub/tests/{store_conformance,queue,worker_routes,record_fixtures}.rs`; `collab-worker/tests/daemon_e2e.rs` gains a `--hub` variant (seeds through the hub's RPC, starts the hub in-process via `collab_hub::app::serve(cfg)`, compares with `simulate` bit for bit), `#[ignore]` until Phase 2's adapters exist.

Verify: `cargo test -p collab-hub`; `cargo test -p collab-engine` (the generic suite still passes over `DirStore`); `cargo clippy --workspace --all-targets -- -D warnings`; `collab-hub check` on an

empty root reports what it lacks; `collab-hub import DIR` loads a `simulate --synthetic --out DIR` tree (every record and object), the first use of the writer path and a handy fixture loader.

Phase 2: worker changes (RSC, `worker/crates/worker/`)

2.1 `daemon/hub.rs`: the adapters

- `HubStore: Store + StreamLeases` with the structure of `DdbStore` (`daemon/store.rs`): the same `Cancel`, `Note`, `api_retry`, lease bookkeeping and the mapping table in its doc comment, with the `tables` field replaced by a `HubRpc` client (ureq, JSON, the `child.rs` op shapes). `complete_prepare`, `commit_checkpoint`, leases and `publish_gold` keep going through `ApiPort` (`HttpApi` at `{hub}/api`), so `prepare.rs`, `accumulate.rs`, `gold.rs`, `jobs.rs` are untouched.
- `HubObjects` = `LocalBlobs` from `daemon/local.rs` at `{hub}/hub/v1/objects`, the loopback rule replaced by "the configured hub origin"; wrapped in `S3Objects` (`daemon/objects.rs`) so `reg/` packing and `refresh_read` bookkeeping stay.
- `HubQueues` = `LocalQueues` with the prefix check against `{hub}/hub/v1/queue/`.
- `connect(cfg)` builds a `Runtime` like `local::connect`; `daemon/start.rs` chooses by backend kind (`aws | local | hub`). A `hub` cargo feature pulls only `ureq`, so a volunteer build can be `--no-default-features --features hub,synthetic` and skip the AWS SDKs and their C toolchain.

2.2 `daemon/config.rs`

New names (in `NAMES`, `docs/runbooks/worker-config.md`, and `backend/terraform/worker-config.tf` as "not set on AWS" rows, because `terraform test` holds the lists equal): `COLLAB_HUB_URL` (set means the worker speaks hub), `COLLAB_HUB_CERT_SHA256` (optional pin), `COLLAB_IDLE_POLICY` (`always|when_idle`), `COLLAB_BUSY_FILE`, `COLLAB_WORK_WINDOW`, `COLLAB_WORKER_TOKEN_FILE` (0600; the token never sits in the TOML). Queue URLs, bucket and table names are not required with a hub; `config check` says so and refuses a mixed configuration.

2.3 `collab-worker enrol <hub-url> --code XXXX-XXXX [--worker-id NAME] [--dir DIR]`

`src/enrol.rs`: `POST {hub}/api/hub/v1/enrol {code, worker_id, os, arch, version, contract, ram_gb, cores}` → `{worker_token, hub: {url, public_url, cert_sha256?}, xpsd_dir, scratch_dir_hint, scratch_cap_gb_hint, min_contract}`; writes `collab-worker.toml` and `worker-token` (0600) into the config dir, runs `config check`, prints the service-install command. Idempotent: a second enrol replaces the token and the hub revokes the old one. The worker id defaults to the hostname, lower-cased and filtered to `[a-z0-9-]`.

2.4 Heartbeat capabilities, catalogue state, version handshake, idle scheduling

- `capabilities.rs::detect` gains `contract` (set by `build.rs` from `docs/api/openapi.yaml` `info.version`, as the uploader does), `catalogue` and `catalogue_path_ok` from `housekeeping.rs::probe_xpsd(dir)` (directory exists, at least one `gdr3-*.xpsd`, one band file opens and parses), re-probed every 5 min so an unmounted share shows up; `busy` and `idle_policy`.

- **Windows RAM detection** (the runbook's known gap): the `sysinfo` crate (pure Rust, no `unsafe` of ours) replaces the `sysctl` shell-out on every OS.
- **Version handshake:** on 409 `contract_mismatch` the heartbeat logs `min_contract`, drains, and the run exits 3; service units treat 3 as "do not restart" (`RestartPreventExitStatus=3`; the launchd wrapper and the Windows task map it the same way).
- **Pause while imaging** (`run.rs`): a `Gate` the prepare and heavy loops consult before each receive; `when_idle` and `busy` means no polling (sleep 15 s); a job in flight finishes (prepare children are short; accumulate and gold release at the next lease boundary as a drain does). `busy` comes from any of: `COLLAB_BUSY_FILE` (touched by the Starfront desktop while a sequence runs), a process check for `NINA.exe` (the volunteer page's "pause while N.I.N.A. is running"), or a local-time window `COLLAB_WORK_WINDOW=08:00-18:00` (the uploader's `--upload-window` parser, 40 lines, moved to shared code). The heartbeat reports the effective state so the fleet view explains an idle worker.
- **Scratch sizing:** `enrol` proposes `scratch_cap_gb = min(free * 0.5, 200)` and refuses below 20 GB (about 2 GB per job plus checkpoints); `config check` warns when the cap exceeds free space.

2.5 Service install (`src/service.rs`)

`collab-worker service install|uninstall|start|stop|status [--dir DIR]`:

- **macOS:** the LaunchAgent and wrapper exactly as `worker/scripts/launch-agent.sh` writes them (`KeepAlive.SuccessfulExit=false`, `ExitTimeOut 150`, `caffeinate`, log to `DIR/worker.log`); the script is reduced to calling the binary.
- **Linux:** a `systemd --user` unit from the runbook's template with `ExecStartPre=config check`, `Restart=on-failure`, `RestartPreventExitStatus=3`; `loginctl enable-linger` printed as the one manual step.
- **Windows:** the default is a **Scheduled Task at logon** (`schtasks /create /sc onlogon`): it runs in the volunteer's own session, so it sees the user's mapped drives and credentials for the SMB share and needs no password. `--as-service` registers a real service through the `windows-service` crate for machines with an administrator who wants one (a service account must be able to read the share, which the runbook explains). The "Windows service MSI" of the earlier plan is not needed.

2.6 Tests

`tests/hub_ports.rs` (mirrors `tests/local_ports.rs`: hub in-process; `HubStore`, ranged downloads, packing round trip, queue visibility and stale receipts, `enrol` used once, wrong code, expired); `tests/daemon_e2e.rs --hub` un-ignored (one worker clean, with crashes, two workers); `tests/cli.rs` (`enrol --help`, `service --help`, `config check` with `COLLAB_HUB_URL` set and the AWS names absent).

Verify: `cargo test -p collab-worker` on macOS and Linux; on Windows at least `cargo test -p collab-worker --no-default-features --features hub,synthetic --test cli --test hub_ports`. By hand on the home network: `collab-hub run` on the Mac; `collab-worker enrol` on the Mac and on the Windows PC (XPSD over SMB); `collab-hub status` shows two healthy workers with `catalogue: ready`; unmount the share on Windows and see `catalogue: missing` within 5 min; touch the busy file and see the worker stop polling.

Phase 3: the hub as a complete standalone backend (RSC)

3.1 Frame ingress over the hub's own HTTP (`routes/frames.rs`)

Port `frames-init` and `frames-complete` over `SqliteStore` (T1, resume, duplicate, sentinel, quota reservation, T2, T2b, size check, prepare send) with the multipart rules of `backend/local/lib/s3-http.js`:

- `init` creates `uploads(upload_id = 32 random bytes hex)` and answers `part_urls[i] = "{public_url}/hub/v1/parts/{upload_id}/{n}"`, `part_size` 16 MiB, `completed_parts` from `upload_parts`, `expires_at` +1 h.
- `PUT /hub/v1/parts/{upload_id}/{n}`: the body is written at offset `(n-1) × 16 MiB` of `objects/raw/{p}/{u}/{frame_id}.{ext}.part`, so nothing is concatenated at `complete`; size checked against `partSizeOf(size, n)`; ETag = `"<md5 hex>"`, stored. Re-upload overwrites. Only an `uploading` frame's upload id is valid; expired uploads (3 days) are aborted by the sweep.
- `complete`: parts complete and ETags match (`bad_etags`, `upload_incomplete { outstanding_parts}`), total size equals `size` (`size_mismatch` → abort and quota release), the first 2 880 bytes pass a FITS/XISF magic check, `rename` `.part` → final, T2/T2b, `prepare` message.
- `library/frames/init|complete|withdraw` the same way under `lib/`.

3.2 Contributor routes (`routes/projects.rs`)

`resolve-join-code`, `projects/{p}` (id or slug), `join` (display-name sentinel, terms, filter mapping, `approval` policy), `membership` GET/PUT, `rigs` GET/POST/PATCH (the `shared/rigs.js` fingerprint rule, in `contract-examples.json`), `filters/normalize` (`collab-filters` directly), `frames` list/get/withdraw (`shared/withdrawal.js`), `status` (`{ok, status_doc}`; a private project's `/private/…` is served to members through a 1-hour HMAC-signed `?sig=` URL), `masters` (same signed URLs), `events`. `/public/filters/aliases.json` from `collab-filters`' JSON export.

3.3 Admin and enrol surface (`routes/admin.rs`), used by the desktop wizard

Authenticated by `rscu_` tokens with scope `admin` (`collab-hub token --admin` prints one once; the desktop stores it in `settings.json` like `collab.adminToken`), never by a cookie (the hub has no web UI). Routes: `POST /api/hub/v1/enrol-codes {kind: worker|uploader, note, ttl_s}` → `{code: "XXXX-XXXX", expires_at}`; `POST /api/hub/v1/enrol` (unauthenticated, code-gated, rate-limited per address); `GET /api/admin/fleet`, `GET|PUT /api/admin/pause` as the contract has them; `GET /api/admin/projects` (queue depths, DLQ counts, disk use); `POST|PUT /api/hub/v1/projects` for a standalone hub without Starfront (the desktop creates projects there); `GET /api/hub/v1/status` (health, versions, disk, catalogue, Starfront link); `GET /hub/v1/worker-manifest` (4.6); `DELETE /api/admin/workers/{id}` (revoke a worker).

3.4 Status publishing

Workers publish exactly as today (`collab_engine::pipeline::publish`, `status::build_status`) into the hub's objects through `HubObjects`. `routes/public.rs` serves `GET /public/{project_id}/{path}` from `objects/public/` with the `Cache-Control` values of `s3-layout.md` (60 s; deep zoom immutable for a year), `Content-Type` by extension, `..` refused. A `PUT` of `public/{p}/status.json` is the publish signal that queues a push to the Starfront server (Phase 4).

3.5 Background tasks (`tasks/`), tokio intervals, each a port of its Lambda

Task	Period	Source
`sweep`	5 min	`backend/lambdas/sweep/index.js` whole (prepare and refresh resend, accumulate resend, stranded streams, `start_after` jobs, expired uploads), same budgets
`gold_scheduler`	hourly	`gold-scheduler/index.js` + `shared/gold-jobs.js`
`retention`	daily and on demand	`retention-sweeper/index.js` (withdrawn raw after 7 d, archived projects after 60 d unless `retain_raw`, library objects, queue retention, `ttl_epoch` rows); the DLQ handlers (`prepare-dlq` → T8/T8r, `gold-dlq` → job failed) as a 5-min DLQ drain; **plus the hub-only rule:** below `limits.free_gb_floor`, prune the `reg/` cache oldest-first by the 45-day rule (gold rebuilds what it needs, open point 83) and say so on the status page
`catalogue`	on project create or grid change, and on request	3.6
`starfront_sync`	15 s pull; push on publish	Phase 4

No `notify` queue and no `notifier`: notifications are Starfront's (Discord, 4.7). The hub writes `events` rows.

3.6 The catalogue service (`catalogue.rs`)

The hub runs the cone search. It has the XPSD path, it is one machine, the result is per project and per `grid.spec()`, and the 45 GB walk over SMB happens once per grid revision instead of once per worker. `GridCatalog::load_or_build(cache, grid, xpsd_dir, CatalogParams::default())` from `collab_engine::project` (what `daemon/catalog.rs::XpsdCatalogs` calls) runs in `spawn_blocking`, one build at a time, cached at `<data_root>/catalogues/catalog-{project}-{spec_hash}.json`. `GET /hub/v1/projects/{p}/catalogue?grid_rev=N` streams it (ETag = spec hash) or answers 202 `building` with `retry_after_s`. Worker side: `daemon/catalog.rs` gains `HubCatalogs: CatalogSource` that fetches into `catalog_file(cache_dir, project)` (the children load that file unchanged), falls back to `XpsdCatalogs` when the hub answers 404 `no_catalogue` (a hub without XPSD) and `PUT` s the built file back. The hub rebuilds on `grid_rev` change and keeps the old file until the new one is ready. The hub never serves XPSD files, only the derived star list of a few thousand rows.

3.7 `collab-hub simulate` and `scripts/hub-e2e.sh`

- `collab-hub simulate --port 0 [--synthetic] [--data DIR]`: a hub on a free loopback port with a temp data root, the synthetic scenario's project, rigs and members seeded through the real routes, `{"url", "admin_token", "enrol_code"}` on stdout, `--exit-on-stdin-close` as the local stack has. It is what `check-contract.mjs` drives and what the e2e script starts.
- `scripts/hub-e2e.sh [--only one|two|kill|all]` (modelled on `local-e2e.sh`): build, start `simulate`, enrol one or two daemons, upload `collab-worker synth-frames` output with **`collab-uploader`** (the real multipart client, `--api-base http://127.0.0.1:PORT/api`, a `rscu_` from `collab-hub token`), wait for the queues to drain and a gold, compare the hub's `objects/` with

`simulate`'s output bit for bit (`backend/local/e2e.mjs`'s checks rewritten as `collab-hub e2e-check` so the script needs no node), and run `website/scripts/browser-smoke.mjs --sim <hub>/objects/public` when Chrome exists. Output under `~/Library/Caches/collab-hub-e2e/`, never `/tmp`.

Verify: `node scripts/check-contract.mjs` with the hub component green; `scripts/hub-e2e.sh --only all`; the uploader's `tests/e2e_local_stack.rs` gets a sibling `e2e_hub.rs`; by hand, the author's real Elephant's Trunk subset through `collab-uploader` to the Mac hub with the Windows worker preparing against the Mac's SMB share, previews rendered by the website smoke from the hub's `public/`.

Phase 4: the Starfront server (SF, `server/`)

4.1 Tables (`server/store.py` SCHEMA, migrations in `Store.__init__`)

```
hubs            (id PK, owner, name, token UNIQUE, grant_secret, lan_url, created, seen, status JSON, cursor)
stack_projects  (project_id PK, sf_project UNIQUE, hub, slug UNIQUE, join_code UNIQUE, rev, doc)
stack_members   (project_id, user_id, rev, doc, PK(project_id, user_id))
stack_rigs      (user_id, rig_id, rev, doc, PK(user_id, rig_id))
stack_frames    (frame_id PK, project_id, user_id, status, sha256, size, path, upload_id, rev, received, relayed, doc)
stack_parts     (upload_id, part_number, size, etag, PK(upload_id, part_number))
stack_library   (frame_id PK, user_id, rig_id, status, path, rev, doc)
stack_changes   (seq PK AUTOINCREMENT, hub, kind, key, at)      -- the pull feed
stack_identities(user_id PK, collab_user_id UNIQUE, display_name)
stack_notified  (project_id, event, key, at, PK(...))
```

`rev` columns and `stack_changes` give the hub a cursor feed. `projects.payload` gains `stacking: { backend: "cloud"|"hub"|"", joinCode, apiBase, hubId, collabProjectId}`; `stackingJoinCode` stays as the cloud case's field for 1.0.0.33 plugins (7.3 maps old to new on read).

4.2 The hub link (`server/hubs.py`, a router over an injected store like `files.py`)

- `POST /api/v1/hubs` (person with the `may_start` role) → register; `GET /api/v1/hubs` (mine); `DELETE` (revoke). `POST /api/v1/hubs/{id}/heartbeat` (hub token) every 60 s with the hub's status JSON, so the Collab tab and "My projects" can say "stacked by *Eddie*'s *hub*, 2 workers, 14 frames queued".
- `GET /api/v1/hubs/{id}/pull?since=` long-polls up to 20 s and returns `{changes, seq}` in `records.md` shapes.
- `GET /api/v1/hubs/{id}/frames?state=uploaded&limit=`; `GET …/frames/{frame_id}/bytes` with `Range` (`FileResponse` honours it); `POST …/frames/{frame_id}/ack {sha256}` → `relayed`. Bytes are deleted after `STARFRONT_STACK_RELAY_KEEP_DAYS` (default 7; 0 = at ack) by the existing sweep thread, unless the project has `collectFiles`, in which case the file is also registered in `files` (same door; the coordinator sees it on `/files`). `KEEP_FREE` from `files.py` applies; `init` answers 429 `quota_exceeded` when the volume has no room.
- `POST /api/v1/hubs/{id}/push` (hub token, multipart): `records` (frame verdict summaries: status, reject and quarantine reasons, a metrics subset, integration seconds; project grid and `grid_rev`; stream stats) and `files` (`status.json`, PNGs, `panels.json`, deep-zoom tiles as a tar, masters under `masters/`) written to `STARFRONT_DATA/stack/public/{project_id}/…` by atomic rename, under a per-hub byte budget.

4.3 The project binding and the listing

- `ProjectRequest` / `ProjectUpdateRequest` gain `stacking: {backend, joinCode, apiBase, hubId}`; `cloud` needs a join code (the existing `_stacking_join_code`), `hub` needs a hub the caller owns or is listed on (`allowedCoordinators` on the hub). Binding mints the `stack_projects` row: `project_id = prj_ + 20 Crockford`, `slug` from the name, grid from the region (`rotation_from_first_light: true`), filters from `requirements`, policy defaults from the contract (`server/stack/defaults.py`, with a test that reads `nina/docs/api/openapi.yaml` when present), `approval: open`, `visibility: unlisted` by default. A change-feed row tells the hub.
- `open_projects` adds `stacking: {backend, hubId, hubName, hubLanUrl, apiBase, collabProjectId, grant}` under the same secrecy rule as the join code today (`grant` only to telescopes on the collaboration or owned by its owner); the old fields stay.

4.4 The front door (`server/stack/`, mounted at `/stack/api`)

A FastAPI router implementing the contributor subset in the Collab envelope, mirroring `/api` paths so the plugin's `BaseUri = https://collab.starfront.space/stack/api/` works: `envelope.py` (`ApiError`, the error table; never 403/404), `auth.py` (`sfg_` HMAC verification against `hubs.grant_secret`; the `starfront_web` cookie for dashboard reads), `projects.py` (resolve-join-code; project; join with display-name uniqueness, terms, mapping; membership), `rigs.py` (the fingerprint rule), `frames.py` (init, resume, duplicate, complete, list, get, withdraw over `stack_frames` and `stack_parts`; the part `PUT` at `/stack/s3/{upload_id}/{n}` written at offset as in 3.1; `list` and `get` answer from the mirror, so verdicts appear once the hub has pushed), `library.py`, `header.py` (the contract's `InitHeader` allow-list; 400 `header_rejected`). `filters/normalize` is not implemented here (the plugin does not call it; it uses the bundled normaliser) and the profile says so. `/stack/public/filters/aliases.json` is a vendored copy of the alias table, the same bytes as the plugin's Core copy.

4.5 Pages and the dashboard

- Static files under `server/web/stack/`, ported from RSC `website/`: `project.html`, `filter.html`, `combined.html`, `dashboard.html`; `js/pages/{project,filter,combined,dashboard}.js`, `js/{status,format,dom,route,svg-charts,deep-zoom,panels-doc,config,session}.js`, `js/components/*`, `js/vendor/openseadragon/`, `css/*`, and a reduced `api.js` (only `GET projects/{p}`, `GET projects/{p}/status`, `GET projects?scope=mine`, `GET projects/{p}/frames?mine=1`, `GET rigs`, `GET auth/me`). Served by `app.mount("/stack", StaticFiles(...))` plus five explicit routes (`/stack/p/{slug}`, `/stack/p/{slug}/filter/{F}`, `/stack/p/{slug}/combined`, `/stack/dashboard`, the home redirect), the port of `website/cloudfront/rewrite.js`. `site-config.js` is generated (`apiBase: "/stack/api"`, `publicBase: "/stack/public"`, `siteName: "Starfront"`), and `tokens.css` is re-accented to Starfront's mint on `#0d1017`.
- `GET /stack/public/{project_id}/{path}` serves the pushed artefacts with the hub's cache headers.
- The dashboard signs in with the existing `/files/signin` (Discord, `starfront_web` cookie); `auth/me` answers from the cookie; "My projects" lists the user's `stack_members` rows with the hub's heartbeat status; "My frames" is `dashboard.js`'s frames table over the mirror (verdicts, CSV). No tokens and no library section in v1.
- **CSP** on `/stack/*` only: `default-src 'self'; script-src 'self'; style-src 'self'; style-src-attr 'unsafe-inline'; img-src 'self' data: blob:; connect-src 'self'; frame-`

`ancestors 'none'` (the `cloudfront.tf` values minus Aladin). `_DOWNLOAD_PAGE`, `/files` and `/sky` keep their inline styles outside it. `tools/check_csp.py` greps `server/web/stack/` for `<script>` bodies, `on*=` and `style=` as `tests/security-headers.tftest.hcl` does. `README.md` replaces "there is not going to be a web interface" with the decision: read-only pages and a cookie dashboard are served; the admin token still never reaches a browser.

- **Remove `server/stacks.py`.**

4.6 Worker-binary manifest hosting

`server/downloads/collab-worker/latest.json`, `latest.json.sig` and per-OS archives, served by `/download/collab-worker/{name}` under a regex as strict as `_SETUP_FILE` (`collab-worker-<semver>-<macos-arm64|windows-x64|linux-x64>.(zip|tar.gz)`, and the same for `collab-hub`). The manifest is rapid-stacker's shape (`crates/gui/src/update/manifest.rs`: `{version, assets: {os: {primary: {url, size, sha256, filename}}}}`) plus `contract`. The hub mirrors it hourly (`GET /hub/v1/worker-manifest`) so a LAN volunteer installs without the VPS. `latest.json.sig` is an Ed25519 signature by the author's offline key (`worker/scripts/sign-manifest.sh`, the shape of `photon-ledger-website/scripts/gen-ed25519.sh`); the public key is embedded in the Starfront desktop and in `collab-worker` (for a later `self-update`). Builds come from a `worker-release.yml` in RSC (cargo-packager is not needed: plain archives with the binary; Windows Trusted Signing and macOS notarisation as rapid-stacker's workflows do them, otherwise unsigned with a note on the volunteer page).

4.7 Notifications

`server/notify.py`: a collaboration's `discordWebhookUrl` (owner-only field) receives `gold_published` ("Gold integration 3 published for Ha: 12.4 h from 4 contributors") and `member_joined`, de-duplicated through `stack_notified`. This replaces SES and `notifier`.

4.8 `tools/check_server.py` and `tools/check_stack_relay.py`

A `# ---- live stacking: hubs` section in the existing style (`case(name, ok, detail)`, real HTTP, fake Discord): register a hub (owner; a non-owner cannot); heartbeat; bind a project and see `stacking.grant` only on the owner's and a joined telescope; verify a grant and refuse a tampered one; `resolve-join-code` → `join` (name uniqueness) → `rigs` (fingerprint equals the contract example) → `frames/init` → PUT two parts out of order → `complete` (size mismatch, duplicate and resume paths) → the frame appears in `hubs/{id}/frames`, bytes by `Range`, `ack` deletes at 0 days; a push with `status.json` and a PNG is served under `/stack/public/…` with `Cache-Control`; `frames?mine=1` shows the pushed verdict; a `collectFiles` project also lists the frame on `/files`; the dashboard answers 401 without the cookie; the CSP header is on `/stack/p/x` and absent on `/download`; hub A's grant is refused by hub B's secret. `check_stack_relay.py` runs a fake hub (a few FastAPI routes) against the server to exercise the pull and push loop end to end.

4.9 `server/run_local.py --lan`

Today it binds loopback only and the notes document an SSH tunnel. Add `--lan` (binds `0.0.0.0`; refused when the data folder is a real server's or holds a `discord.env`; prints a loud warning and the LAN URL) so the Windows PC reaches the stand-in server directly. The tunnel stays as the alternative.

Verify: `check_server.py`, `check_stack_relay.py`, `check_csp.py`, `check_local_server.py`; `run_local.py --lan` on the Mac, the hub registered from `curl`, `collab-uploader` → `http://mac:8800/stack/api` → the hub pulls, stacks, pushes → `http://mac:8800/stack/p/<slug>` renders in a browser; `browser-smoke.mjs --sim` against `STARFRONT_DATA/stack/public/<project_id>`.

Phase 5: the Starfront desktop (SF, `astrocontrol/`)

5.1 Settings (`config.py` `DEFAULTS`; unknown keys are dropped on save)

```
"hub": {"enabled": False, "binary": "", "configDir": "", "dataRoot": "", "catalogueDir": "",
        "bind": "0.0.0.0:8790", "publicUrl": "", "adminToken": "", "autoStart": True,
        "registeredId": "", "registeredName": ""},
"volunteer": {"enabled": False, "hubUrl": "", "workerId": "", "idlePolicy": "when_idle",
              "pauseWhileSequencing": True, "pauseWhileNina": True, "workWindow": "",
              "scratchDir": "", "scratchCapGb": 0, "installedVersion": "", "manifestUrl": ""},
```

5.2 `astrocontrol/hub.py`: supervising `collab-hub`

`HubSupervisor` (the `desktop.ServerThread` shape with `subprocess.Popen`): writes `collab-hub.toml` from the settings, runs `collab-hub check` and shows its report, starts `collab-hub run --log-format json`, tails the log into a ring buffer, polls `GET /api/hub/v1/status` every 10 s, restarts on crash with back-off, keeps the Mac awake with `caffeinate` while running, stops on exit. Registration with the Starfront server uses the existing `collab.userToken` through `collabadmin.py`'s helper and stores `registeredId`. Enrol codes are minted through the hub's admin surface and shown in the tab. The hub binary comes from the same signed manifest as the worker (5.3), installed under the data root's `bin/`.

5.3 `astrocontrol/volunteer.py`: "Volunteer this machine for live stacking"

Fetch `latest.json` and `.sig` (hub first, Starfront server second); verify the Ed25519 signature (`cryptography` added to `requirements.txt` and the PyInstaller spec; the server install does not need it; a pure-Python RFC 8032 verifier is the fallback if bundle size bites); download this OS's asset to a temp file; verify `sha256`; unpack into `<data_root>/collab-worker/<version>/`; run `collab-worker enrol <hub> --code <code>` (typed by the user, or fetched automatically when this machine also runs the hub); run `collab-worker service install`; show `service status` and the last heartbeat from the hub's fleet route. "Update" repeats the download and restarts the service (copy to a temporary name then rename, the LaunchAgent script's binary-swap rule). `sequencer.py` touches `<data_root>/collab-worker/busy` at sequence start and removes it at the end; `volunteer.py` writes `COLLAB_BUSY_FILE` into the worker's TOML.

5.4 Routes in `main.py` (flat, like the rest)

`GET /api/hub`, `POST /api/settings/hub`, `POST /api/hub/start|stop|check|register|enrol-code|install`, `GET /api/hub/log`; `GET /api/volunteer`, `POST /api/settings/volunteer`, `POST /api/volunteer/install|update|uninstall|start|stop`, `GET /api/volunteer/status`. The `/ws` snapshot gains `hub` and `volunteer` summaries. `/api/collab/projects` create and update pass the `stacking` block through.

5.5 UI (`web/index.html`, `web/collab.js`)

In the Collab tab: the "Start a collaboration" form gets a **Live stacking** fieldset (backend *None* / *Rapid Stacker Collab (cloud)* / *My hub*; join code for cloud, hub picker for hub; visibility public or unlisted). Two new collapsible sections: **Live stacking hub** (wizard: data-root and catalogue-folder pickers via `Bridge.pick_folder`, the `check` result inline, bind and public URL, Start/Stop, register with Starfront, "New enrol code", fleet table from the hub's fleet route, log tail) and **Volunteer this machine** (hub URL or "this machine's hub", enrol code, install/update/status, idle-policy switches). Local mode is `collab.serverUrl` pointed at the local server (it already exists); the hub section shows which server it is registered with. `check_controls.py` must pass.

5.6 Checks and docs

`tools/check_hub.py` (a fake `collab-hub` script answering `check` and `run` with a tiny status route; the supervisor starts, polls and restarts it), `tools/check_volunteer.py` (a fake manifest server with a good and a bad signature; the installer refuses the bad one and never runs an unverified binary; the fake `collab-worker` records the `enrol` and `service install` argv). README: a "Live stacking at home" section with the design decisions (why the hub pulls from the server; why a per-hub grant; why the Windows worker is a logon task; why the catalogue folder is the user's own copy; what a volunteer's machine sees).

Verify: `check_controls.py`, `check_hub.py`, `check_volunteer.py`, `check_server.py`; by hand on the Windows PC: Volunteer page → install from the Mac hub's manifest → `collab-worker` running as a logon task, visible in the Mac's fleet table; pause while N.I.N.A. is open; `packaging/build.ps1` produces an installer that includes `cryptography` and the new pages.

Phase 6: the N.I.N.A. plugin (SF, `nina/`)

6.1 Models and settings

- `Server/Models.cs` `ProjectListing` gains `[JsonProperty("stacking")] StackingBinding Stacking` (`Backend`, `HubId`, `HubName`, `HubLanUrl`, `ApiBase`, `CollabProjectId`, `Grant`); `StackingJoinCode` / `StackingLinked` stay for old servers.
- `Settings/StarfrontSettings.cs`: `StackingApiBase` stays the cloud base; add `StackingHubTrustLan` (default true) and `StackingDefaultBackend` (`cloud|hub`) for rig- and library-scoped calls. Grants are not settings; they live on the link.
- `Services/Stacking/StackingLinks.cs` `StackingLink` gains `Backend`, `ApiBase`, `LanApiBase`, `Grant`, `GrantExpiresUtc`, `HubId`.

6.2 Clients and routing (no Core change)

- `StackingClients` becomes a cache: `For(link)` → an `ApiClient` per `(apiBase, tokenSource)`; cloud links use `SettingsTokenStore` (`rscu_`), hub links a `GrantTokenStore` returning the link's current grant (refreshed by the joiner at each check-in; `Clear()` marks the link `Failed` so a stale grant is re-fetched, not retried).
- `RoutingApiClient : IApiClient` (`Services/Stacking/RoutingApiClient.cs`): every project-scoped call is dispatched by its project argument to the link's client; rig- and library-scoped calls go

to `StackingDefaultBackend`; `DeviceStart/Poll` and `RevokeOwnToken` go to the cloud client. `UploadEngine` and `VerdictPoller` receive the router, so Core stays verbatim.
- LAN short-cut: a `LanProbe` checks `{HubLanUrl}/api/hub/v1/health` (2 s timeout; the answer's `hub_id` must match the listing) at check-in; while it answers, the link's effective base is the LAN URL, else the server's `/stack/api`. Both accept the same grant. A frame started on one base finishes on it (the queue item records the base); a resume after a base change re-inits (`resumed` or `duplicate`, both fine).

6.3 Joining a hub project

`StackingJoiner.ReconcileAsync`: a listing with `stacking.backend == "hub"` and a grant is joined without a device sign-in: `GetProjectAsync` → `UpsertRigAsync` → `JoinProjectAsync` (display name = the Starfront user name; terms accepted headlessly as today's hint says) → `GetMembershipAsync`. `StackingEnabled` stays the master switch; `StackingSignedIn` is only required for cloud links; the options page says "Hub projects need no Rapid Stacker Collab sign-in".

6.4 Core sync note

Nothing above needs a Core change. Two candidates that would go upstream first if wanted later: `ApiClientOptions` taking a `Func<Uri>` so one client could switch base, and a `prefer_relative_part_urls` hint. Record both in `SOURCE.md` as "Local differences: none".

6.5 `nina/MANUAL_CHECKLIST.md` additions

A "Hub projects" section: the listing shows the binding; no Collab sign-in required; a light reaches the server's `/stack/s3` (watch `stack_frames`) and, on the LAN, the hub's `objects/raw`; the verdict appears in the dock within a poll; removing the telescope from the collaboration stops uploads within one check-in and the queue holds the frames; the LAN short-cut falls back when the hub is stopped; mixed cloud and hub links on one night upload each to its own backend; the SQLite preload steps repeated unchanged.

Verify: `cd nina && dotnet build Starfront.sln -c Release && dotnet test tests/Starfront.Collab.Tests` with new tests for `RoutingApiClient` (dispatch table, default-backend rule), `GrantTokenStore`, `LanProbe` (unreachable, wrong id, reachable) and the joiner over a fake `IApiClient`; the checklist on the Windows PC against the local topology.

Phase 7: local test mode, coexistence, CI

7.1 Topology on the home network

Machine	Runs	Address
Mac	`python server/run_local.py --lan --data ~/starfront-local` (stand-in Discord, owner `ncedelliott`)	`http://<mac>.local:8800`
Mac	`collab-hub run` (data root `~/starfront-local/hub`, `xpsd_dir ~/Astronomy/XPSD`, `[xpsd_paths] windows = '\\<mac>\XPSD'`), registered against the local server from the desktop Collab tab	`http://<mac>.local:8790`
Mac	`collab-worker` enrolled (idle policy `always`); `collab-uploader` for synthetic or archived frames	
Mac	the XPSD folder shared read-only over SMB (System Settings → Sharing)	`smb://<mac>/XPSD`
Windows PC	N.I.N.A. with `ServerUrl = http://<mac>.local:8800` and `StackingEnabled`; `collab-worker` installed from the Starfront desktop's Volunteer page against the Mac hub, `xpsd_dir = \\<mac>\XPSD`	
Windows PC	Starfront desktop in local mode (`collab.serverUrl = http://<mac>.local:8800`)	

mDNS is **not in v1**: `_starfront-hub._tcp` would add a resolver dependency on each OS for a URL the user types once and the server hands to every telescope in the listing (`hubLanUrl`). Revisit if a hub ever has to be found by a machine the server does not know.

7.2 Scripts and what each phase's run proves

- RSC `scripts/hub-e2e.sh` (Phase 3): hub, worker and uploader with no Starfront.
- SF `tools/local_night.py`: starts `run_local.py --lan`, the hub and one worker on the Mac from the settings, registers the hub, binds a throw-away collaboration, prints the three URLs and the enrol code for the Windows PC, tails the logs; `--synthetic` uploads `synth-frames` output with `collab-uploader` so a stack appears without a telescope. The one command a reviewer runs.
- `docs/runbooks/starfront-hub.md` (RSC) and README "Live stacking at home" (SF): the topology, the SMB share, firewalls (8790 inbound on the Mac), what to look at. `Running-the-servers-locally.md` at the Astro level is superseded by them.

What proves each phase here: Phases 1–2 the fleet table and `hub-e2e`; Phase 3 `status.json` under the hub's `/public`; Phase 4 the same page at `http://<mac>:8800/stack/p/<slug>` after a push, and `check_stack_relay.py`; Phase 5 the Windows worker installed from the desktop; Phase 6 a real light from N.I.N.A. shown as `accepted` in the dock and in the live preview within a minute, once over the LAN short-cut and once with the hub's LAN port blocked (server relay).

7.3 Coexistence with AWS

- Existing Starfront projects with `stackingJoinCode` read as `stacking.backend = cloud` (`_stacking_of(payload)` in `app.py`); nothing to migrate. Plugin 1.0.0.33 keeps working against new servers; new plugins against old servers see no `stacking` block and behave as 1.0.0.33.
- The AWS stack is untouched except the config-name list in `worker-config.tf`. The worker image keeps `aws` default features. A worker is cloud (`COLLAB_API_BASE` plus AWS names) or hub

(`COLLAB_HUB_URL`), never both.
- A project is bound to one backend for life; the form refuses changing `backend` once a frame exists. Moving a project between backends is out of scope; an export of the live M33 project is a later `collab-hub import --from-aws` if the author wants it.
- Contract versioning: the hub profile (0.A); the hub's `min_contract` is bumped only with a note in the deploy record of the hub release; `/api/v1/health` on the Starfront server reports `stackContract`.

7.4 CI

- RSC: `worker-ci` builds the hub; `hub-e2e` job; `contract-ci` runs the hub component.
- SF: move `nina/.github/workflows/ci.yml` and `release.yml` to `.github/workflows/` (GitHub reads only the root) with `working-directory: nina` and `paths: ['nina/**']` ; add `python-checks.yml` running every `tools/check_*.py` on ubuntu and windows with Python 3.12 (the checks fake hardware and Discord); fix the "Nothing runs CI" line in `CLAUDE.md` when it lands.

---

8. Reuse map

Need	Reuse
Store semantics	`worker/crates/engine/src/store/local.rs` (`DirStore`), `store/mod.rs` traits, `lease.rs`
Objects	`collab_engine::store::local::DirObjects`; worker side `daemon/objects.rs` (`S3Objects`), `daemon/packed.rs`
Worker routes' rules	`daemon/fake/api.rs` (made generic over `&dyn Store`); Lambda references `backend/lambdas/worker-*/index.js`, `shared/prepare-results.js`, `shared/streams.js`, `shared/gold-jobs.js`
Queue semantics	`daemon/fake/mod.rs` and `docs/api/queues.md`
Wire shapes for the RPC	`daemon/child.rs` request and reply enums; `daemon/ports.rs` typed route structs
HTTP clients on the worker	`daemon/local.rs` (`LocalBlobs`, `LocalQueues`, ranged download), `daemon/aws/api.rs` (`HttpApi`)
Catalogue	`daemon/catalog.rs` (`CatalogSource`, `catalog_file`), `collab_engine::project::GridCatalog::load_or_build`, rapid-stacker `crates/core/src/astrometry/xpsd.rs::cone_search_local`
Multipart over plain HTTP	`backend/local/lib/s3-http.js`, `frames-init/index.js`, `frames-complete/index.js`
Scheduled repairs	`backend/lambdas/{sweep,gold-scheduler,retention-sweeper}/index.js`, `shared/project-purge.js`
Capabilities, housekeeping, service	`daemon/capabilities.rs`, `daemon/housekeeping.rs`, `worker/scripts/launch-agent.sh`, the runbook's systemd unit
Pages	RSC `website/js/pages/{project,filter,combined,dashboard}.js`, `js/status.js`, `js/components/*`, `css/*`, `cloudfront/rewrite.js`
Manifest	rapid-stacker `crates/gui/src/update/manifest.rs`, `download.rs` (sha256 check)
Starfront server patterns	`server/files.py` (router over an injected store, `who()`, streaming `PUT`, `KEEP_FREE`, cookie), `server/store.py` (payload JSON, migrations), `server/run_local.py` (stand-in Discord), `tools/check_server.py`
Desktop patterns	`astrocontrol/desktop.py::ServerThread`, `collabadmin.py`, `config.DEFAULTS` sections, `web/collab.js` refresh loop, `tools/check_controls.py`
Plugin patterns	`Services/Stacking/StackingClients.cs`, `StackingLinks.cs`, `StackingJoiner.cs`, `SettingsTokenStore.cs`, `Server/StarfrontClient.cs`, `Server/Models.cs`

9. Risks and open questions

9.1 Risks

Risk	Mitigation
`SqliteStore` drifts from `DirStore` and the Lambdas	the generic store conformance suite, record fixtures, `daemon_e2e --hub` bit for bit against `simulate`, the contract check's drive-for-real; the `DirStore`-embedded fallback of 0.1
The contributor subset exists twice (Rust on the hub, Python on the server)	both held to `docs/api` by `check-contract.mjs` (a `starfront` component reading `server/stack/*.py` routes) and by the shared `contract-examples.json`; it is the price of uploads surviving a sleeping hub
A hub on a laptop sleeps or goes offline mid-night	leases and visibility already cover a vanished worker; for a vanished hub the workers idle and retry, the server keeps relaying frames to disk until the hub returns (`KEEP_FREE` floor), the hub page shows "last seen", and the desktop supervisor runs `caffeinate`
The VPS volume fills with relayed subs	deletion after ack (7 d, 0 for non-`collectFiles` projects), 429 `quota_exceeded` below `KEEP_FREE`, the contract's per-contributor quotas, the `/files` stats
SMB performance and the 45 GB catalogue walk	the cone search runs once on the hub; workers read `catalog-{project}.json` only; `probe_xpsd` detects an unmounted share
Clock skew between hub, workers and server	leases use the hub's clock; the worker uses `renew_by` from the answer; the heartbeat carries `server_time` and the worker logs skew above 30 s; grants carry `exp` from the server's clock with 5 minutes' leeway on the hub
Volunteer machines see other people's raw subs	as AWS home workers do today; stated on the volunteer page and in the README: frames are stripped of observer location (the plugin strips, the header allow-list refuses), live in scratch only for the job, and are collected afterwards; the hub owner decides who gets enrol codes; `private` visibility keeps artefacts off the public pages but not off the workers
Windows RAM detection reports 4 GB and skips gold	`sysinfo`
Worker version skew across a fleet	`contract_mismatch` refuses old workers; the manifest's `contract`; the volunteer page's "Update"; versions in the fleet table
The CSP breaks a ported page	`check_csp.py` plus `browser-smoke.mjs --sim` against the Starfront-served pages
`cryptography` in the PyInstaller build	hidden import in the spec; the pure-Python verifier as fallback
Licence boundaries	`collab-hub` is proprietary like the rest of `worker/`; the Starfront repo (MPL-2.0) receives the contract (docs), the ported pages (the author owns them; add an attribution header; confirm the relicence), the alias table, and binaries as downloads
XPSD licence	the plan points at the user's own PixInsight download, never copies it (no `reference/xpsd/` sync on a hub), and the catalogue endpoint serves derived star lists only; the wizard's help links to the PixInsight download page
Hub backup	`collab-hub backup DIR` (`VACUUM INTO` plus a tar of `objects/masters` and `objects/public`; `raw/`, `reg/` and `live/` are rebuildable or relayed); the wizard offers a nightly path; "the masters are the thing to keep"
Hub admin token in `settings.json`	the desktop stores its other tokens in plain text today; keep parity for v1 and note OS keychain or DPAPI as a later hardening for all three

9.2 Open questions for the author

1. **Library frames with two backends:** one library per backend, routed by `StackingDefaultBackend` (the plan), or the hub library fed from the user's Starfront `/files` darks? The plan does the former and flags the duplication.
2. **Who may bind a project to a hub:** the hub owner only, or any coordinator the owner lists (`allowedCoordinators`, the plan)?
3. **Visibility default** for hub projects: `unlisted` (the plan); `public` would need a directory the Starfront server does not have.
4. **Deletion:** deleting a Starfront collaboration bound to a hub files a Collab-style delayed deletion on the hub (7 days, restorable) and purges the relay copy; the Starfront-side "Close" semantics (keep masters? hand contributors their raw back?) need the author's rule; `docs/runbooks/deleting-a-hub-project.md` records it.
5. `collab-worker self-update` for headless Mac and Linux volunteers (same manifest and signature), or is re-running the install enough for v1?
6. **TLS on the LAN:** off by default (the plan) or on with the pinned self-signed certificate from day one?
7. **Relicensing the ported pages** from proprietary (RSC `website/`) to MPL-2.0 inside the Starfront repo, or keeping them under `server/web/stack/` with a separate notice?

10. Suggestions: what the brief left out

Things not in the original ask that the plan now covers, or that the author should decide on:

1. **Fleet observability.** The hub's fleet route and the desktop's fleet table (worker, version, contract, catalogue state, busy or idle and why, frames per hour, last heartbeat, DLQ counts). Without it a volunteer fleet is invisible and "why is nothing stacking" is unanswerable.
2. **Pause while imaging and work windows** (2.4). A volunteer's PC is usually the one running N.I.N.A.; stacking during capture competes for disk and CPU. Idle policy, the busy file from the sequencer, the `NINA.exe` check and a time window cover it.
3. **Power management.** `caffeinate` for a Mac hub or worker; on Windows the logon task follows the user's power plan, and a sleeping volunteer simply misses messages (the sweep resends them). The hub page shows "last seen" per worker.
4. **Version and contract handshake** (0.A 111, 2.4). A fleet installed from a manifest will skew; the hub refuses old workers cleanly instead of failing mid-job.
5. **Clock skew** (9.1). Leases and grants are time-based; the plan pins them to the hub's and the server's clocks respectively.
6. **Security of volunteer machines** (9.1). What a worker can see, enrol-code rate limiting, worker revocation, per-hub grant secrets, no Discord ids in Collab records, LAN-only bind by default.
7. **Disk budgeting and backup** (3.5, 9.1). A free-space floor on the hub, the `reg/` cache pruned under it, the VPS relay kept small, and a backup command that keeps the masters.
8. **Notifications** without SES: Discord webhooks per collaboration (4.7), which Starfront users already live in.
9. **Deletion and retention runbooks** for hub projects (9.2 item 4), so a closed collaboration does not leave 2 TB on someone's NAS.
10. **A standalone hub** (3.3): a hub with no Starfront server at all (the desktop creates projects on it directly) is the simplest thing to test and is what a club with no internet at the observatory wants.

11. **Licence boundaries** (Findings; 9.1): where proprietary code stops and MPL begins, and the XPSD question.
12. **The plugin's CI** has never run (7.4); moving the workflows is a one-commit fix that this work should carry.
13. **Throughput honesty** (Findings): one capable machine already keeps up with a large night, so the volunteer feature is sold as resilience and gold speed, and defaults are sized for small scratch and politeness, not maximum throughput.
14. **An uploader enrol code** (3.3) so non-N.I.N.A. capture software on the LAN can feed the hub with `collab-uploader` without the cloud device flow.
15. **Not done on purpose:** a GPU path (none exists, open point 91), moving a project between backends, a public directory on the Starfront server, mDNS discovery, an export of the live AWS projects (a later `import --from-aws`).

11. Verification summary

What	Where	How
Contract and hub profile	RSC	`npx @redocly/cli lint docs/api/openapi.yaml`; `node scripts/check-contract.mjs`
Hub core and worker adapters	RSC, macOS and Linux	`cargo test -p collab-hub`; `cargo test -p collab-worker` (incl. `daemon_e2e --hub`, `hub_ports`); clippy `-D warnings`; `check-no-local-patch.sh`, `check-dep-skew.sh`
Standalone hub end to end	RSC	`scripts/hub-e2e.sh --only all`; `website/scripts/browser-smoke.mjs --sim` on the hub's `public/`
Starfront server	SF, macOS	`python tools/check_server.py`, `check_stack_relay.py`, `check_csp.py`, `check_local_server.py`
Starfront desktop	SF	`check_controls.py`, `check_hub.py`, `check_volunteer.py`
Plugin	SF, macOS build; Windows run	`dotnet build Starfront.sln -c Release && dotnet test tests/Starfront.Collab.Tests`; `nina/MANUAL_CHECKLIST.md` "Hub projects"
The whole thing	home network	`tools/local_night.py` then a real night from N.I.N.A. on the Windows PC, once over the LAN short-cut and once via the server relay

Critical files

- `Rust/rapid-stacker-collab/worker/crates/engine/src/store/local.rs` (`DirStore`, the reference for every transition `SqliteStore` ports)
- `Rust/rapid-stacker-collab/worker/crates/worker/src/daemon/fake/api.rs` (the Lambdas' worker-route rules in Rust, to be made generic over `&dyn Store`)
- `Rust/rapid-stacker-collab/worker/crates/worker/src/daemon/local.rs` and `child.rs` (the HTTP port adapters and the RPC shapes the worker's `hub` adapters are built from)
- `starfront/server/files.py`, `server/app.py` (`open_projects`), `server/store.py` (the patterns the front door, relay and dashboard extend)
- `starfront/nina/src/Starfront.Collab/Services/Stacking/StackingJoiner.cs`, `StackingClients.cs`, `StackingLinks.cs` (where the per-project backend, the grant store and the LAN short-cut land without touching the verbatim Core)